

FINM 32950: Intro to HPC in Finance

Lecture 5

Chanaka Liyanarachchi

April 14, 2025

Black Scholes Pricer

Monte Carlo on the GPU

Course Wrap-up

Black Scholes Pricer

Black Scholes Pricer

- ▶ You can structure this program very similar to the previous vector additon example.
- ▶ How do we write the kernel function (to price Options)?
 - ▶ Inputs: arrays of S , K , T , r , v
 - ▶ Output: call price (ignoring put price and greeks)

```
--global__ void BlackScholesKernel(  
    float *S, float *K, float *T,  
    float* r, float* v,  
    float* C)
```

- ▶ For each option, we calculate:

```
d1 = ...;  
d2 = ...;  
nd1 = cdf_normal(d1);  
nd2 = cdf_normal(d2);
```

- ▶ Now we can find the call price and the put price:

```
C[idx] = ...;  
P[idx] = ...;  
}
```

- ▶ A kernel function can use other device functions.
- ▶ E.g. `cdf_normal()`

Device Functions

- ▶ `cdf_normal()` is called from the GPU and executed on the GPU.
- ▶ We define this function using `__device__` qualifier:

```
__device__ float cdf_normal(const float x)
{
    .....
    .....
}
```

- ▶ This kernel and device functions use several math functions.
- ▶ A full list of math functions supported by CUDA can be found at:
<http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#mathematical-functions-appendix>

- For assignment 3, (if you use CUDA) you should populate the inputs on the CPU, similar to what you did for Assignment A.

```
for (int i = 0; i < NumOptions; ++i)
{
    h_S[i] = random_data(...);
    h_K[i] = random_data(...);
    h_T[i] = random_data(...);
    h_r[i]  = random_data(...);
    h_v[i] = random_data(...);
}
```

- And, do the computations on the GPU and copy all results back to the CPU.

Monte Carlo on the GPU

Generating Random Numbers

- ▶ We can use the cuRAND¹ library to generate random numbers in CUDA applications.
- ▶ Code below shows how to generate pseudo random numbers from the uniform distribution:
- ▶ Step 1: create a generator. It takes a generator type as a argument.

```
curandGenerator_t gen;
```

```
curandCreateGenerator(&gen, CURAND_RNG_PSEUDO_MTGP32);
```

- ▶ Step 2: initialize it using a seed:

```
curandSetPseudoRandomGeneratorSeed(gen, 1234ULL);
```

¹<http://docs.nvidia.com/cuda/curand/index.html#axzz4jskVkPyT>

- ▶ Step 3: allocate memory on the device:

```
int N = 1000;  
int numBytes = N*sizeof(float);
```

```
float *d_rands;  
cudaMalloc((float**)&d_rands, numBytes);
```

- ▶ Step 4: generate N random numbers on the device:

```
curandGenerateUniform(gen, d_rands, N);
```

- ▶ cuRAND supports random number generation from the following distributions:

1. uniform
2. normal
3. lognormal
4. poisson

- ▶ Reference: <http://docs.nvidia.com/cuda/curand/device-api-overview.html#distributions>

- ▶ Now we can use these random numbers on the device.

- ▶ Cleaning up:

```
curandDestroyGenerator(gen);
```

- ▶ And, clean up device memory:

```
cudaFree(d_rands);
```

- ▶ See curand_test.cu for details.
- ▶ To build we need to pass curand library as an additional argument:

```
nvcc -lcurand curand_test.cu -o curand_test
```

- ▶ To run:
./curand_test

CUDA Example: Monte Carlo PI

- ▶ We've estimate the value of π using Monte Carlo.
- ▶ Used serial code and parallel code on CPU.
- ▶ The Code below shows how to do this simulation on the GPU.

▶ MC PI kernel (Basic):

- ▶ Uses N points (each point is a pair of random numbers from the uniform distribution). You could use one array (x and y) or two arrays, as shown below
- ▶ result: indicator if each point is in the circle

```
--global__ void pi_kernel(float* x,  
                           float* y,  
                           int* results, int N)  
{
```

▶ Thread index:

```
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
```

▶ Pick x and y points and compute the radius:

```
    float radius = x[idx]*x[idx] + y[idx]*y[idx];
```

▶ Using the radius we can determine if the point is inside the circle.

```
    if (radius <= 1) results[idx] = 1;  
    else results[idx] = 0;  
}
```

▶ See demo for details.

Course Wrap-up

We Discussed

- ▶ This course is about to improve performance using **parallel programming**.
- ▶ We discussed **3 main technologies**:
 1. Vectorization
 2. Multicore
 3. GPGPU
- ▶ We discussed how to optimize and improve performance using a **structured approach**:
 - ▶ measure
 - ▶ profile
 - ▶ experiment meaningful changes
 - ▶ repeat

Vectorization

- ▶ We can get significant speed-ups on modern processors using SIMD:
 - ▶ we can use intrinsics
 - ▶ or, auto vectorization
- ▶ We looked at both techniques.
- ▶ We prefer auto vectorization:
 - ▶ we have to understand when a compiler vectorizes code and when it does not
 - ▶ how to guide the compiler to vectorize code
 - ▶ read vectorization reports

Multicore

- ▶ We discussed basic ideas about multithreading:
 - ▶ using threads to do work in parallel
 - ▶ related concepts: critical sections, race conditions, locks, deadlocks, atomics
- ▶ Solve problems in finance using high level constructs:
 - ▶ OpenMP
 - ▶ TBB
- ▶ Introduced structured parallelism, using parallel patterns.

GPGPU

- ▶ Not an easy topic for a beginner-level course in a short time.
- ▶ We looked at some useful applications.
- ▶ We understand the power of CUDA and its limitations.
- ▶ There are many other interesting and important topics we can look at, but we have to stop here due to time constraints.

Parallel Computing

- ▶ Parallel programming is a broad topic.
- ▶ We have many solutions.
- ▶ Examples we looked at include:

Example	C++ Serial	Python/Cython	Parallel CPU/GPU
Black Scholes	✓	✓	✓
Tree Pricer	✓	✓	✓
MC PI	✓	try it	✓

- ▶ This should provide you with an excellent opportunity to not only learn about each technology but also see the trade-offs and pros-and-cons of each, firsthand.

- ▶ The choice between various technologies often depends on (among other things):
 - ▶ developer expertise/comfort-level
 - ▶ hardware
 - ▶ application requirements
- ▶ Understanding the trade-offs, pros-and-cons of each technology allows you to select the most appropriate tool for your specific use case and your comfort level.
- ▶ It is unrealistic to anticipate that everyone will master these concepts within a span of 5 weeks; acquiring proficiency requires time and practice.
- ▶ The Course objective was to provide an introduction to essential technologies, demonstrating their possibilities and initiating your journey. I trust we have met that objective.

Next ...

- ▶ Apply/use HPC concepts we discussed every time you get an opportunity.
- ▶ Practice and keep learning.
- ▶ Please don't hesitate to contact me if you need any assistance.

THANK YOU

PLEASE KEEP IN TOUCH

PLEASE SEND YOUR FEEDBACK

GOOD LUCK!